

编译系统实践

1. ToyC 编译器实现_Ocaml_C++_JAVA

任务概述

四人为一组，实现 ToyC 语言（见 [ToyC 语言定义](#)）的编译器，将 ToyC 源文件经过词法分析、语法分析、语义分析、代码生成等步骤，编译成能够正确执行的 RISC-V32 汇编代码，并完成一份实践报告。编译器实现将由在线评测系统进行自动测试并评分。

实现要求

你可以使用 C++（C++20）或 Java（JDK 21）或 OCaml（5.3.0）中的任意一种语言来实现编译器。评测时的编译时间限制很宽松，且编译器本身的执行效率不影响评分，所以建议选择你最擅长的语言来完成。鼓励编译器中实现多种优化算法（如公共子表达式消除、死代码删除、寄存器分配优化等），以提高所生成汇编代码的运行效率。禁止抄袭代码或套用 Clang、LLVM 等现成框架。

可以使用的构建系统：

- C++：CMake（4.0.3）、Make
- Java：Maven（3.9.10）、Gradle（8.5）
- OCaml：Dune（3.19）

可以使用的第三方库：

- C++：Flex、Bison/Yacc、ANTLR（4.13.1）
- Java：ANTLR（4.13.1）
- OCaml：ocamllex、ocamlyacc、Menhir

接口说明

编译器需要从标准输入流（stdin）中读取源程序代码，最终向标准输出流（stdout）中写入汇编程序代码。标准错误流（stderr）的输出随意。

提示：本地测试时，为了方便，你可以重定向标准输入输出流，用文件读写的方式来输入输出：

```
/path/to/compiler < /path/to/input_file.tc > /path/to/output_file.s
```

在性能测试中，编译器将接受一个参数 `-opt` 表示开启优化。你可以选择在接受该参数时启用某些优化算法，也可以选择忽略它。

代码提交与评测

你需要将编译器项目上传到 Git 仓库。提交评测时，你需要提供仓库地址、访问令牌和分支名（可选），例如：https://myuser:ghp_XXXXXX@github.com/myuser/myproj.git

访问令牌的获取方法详见：

- [GitHub 访问令牌获取方法](#)
- [Gitee 访问令牌获取方法](#)
- [GitLab 访问令牌获取方法](#)

提交后，评测系统将克隆指定分支的代码到评测环境，自动检测编程语言、项目结构、构建系统、可执行文件名或主类名等，并进行构建和评测。

项目中需要包含构建系统配置文件（如 `CMakeLists.txt`、`Makefile`、`pom.xml`、`build.gradle`、`dune-project` 等）。除项目外，每组还需要提交一份实践报告文档。

测试用例与评分标准

所有测试用例的 ToyC 源文件均无语法错误，符合该语言的所有语义约束，且不包含未定义行为。源文件编译成的可执行文件能在有限时间内执行完成。程序的运行结果以主函数的返回值（即程序的退出代码，是 0~255 之间的整数）为准。当生成的代码在规定时间内完成运行，且运行结果与正确结果一致时，测试用例视为通过。

单个测试点的时间限制为：

- 功能测试：1s1s
- 性能测试：20s20s

最终评分将综合考虑测试用例的通过率（功能）、生成代码的运行时间（性能）和实践报告内容。为了进一步鼓励后端优化，今年提高了性能分数的占比。每个测试用例的性能得分将以 gcc -O2 生成代码的运行时间作为基准进行计算，以满分封顶：

总得分 = 评测得分 × 80% + 实践报告得分 × 20%
评测得分 = 功能得分 × 75% + 性能得分 × 25%

功能得分 = $\frac{\text{通过的功能测试用例数量}}{\text{功能测试用例总数量}} \times 100$

性能得分 = $\sum \text{性能测试用例} \min\{1, \frac{\text{基准时间}}{\text{实际运行时间}}\}$
性能得分 = $\frac{\sum \text{性能测试用例} \min\{1, \frac{\text{基准时间}}{\text{实际运行时间}}\}}{\text{性能测试用例总数量}} \times 100$

ToyC 语言定义

ToyC 是 C 语言的一个简化的子集，为本课程实践的练习而设计。相较于 C 语言，ToyC 省去了数组、指针、I/O、多文件编译等特性。ToyC 源文件可以用 C 编译器直接编译成可执行文件，其运行结果与等同的 C 代码编译成的可执行文件一致。

ToyC 语言的文法

ToyC 语言的文法如下，其中 `CompUnit` 为开始符号：

编译单元 $\text{CompUnit} \rightarrow (\text{Decl} \mid \text{FuncDef})^+ \text{声明}$
 $\text{Decl} \rightarrow \text{ConstDecl} \mid \text{VarDecl}$ 常量声明
 $\text{ConstDecl} \rightarrow$
 “const” “int” ID “=” Expr “;” 变量声明
 $\text{VarDecl} \rightarrow$ “int” ID “=” Expr “;” 语句
 $\text{Stmt} \rightarrow \text{Block} \mid$
 “;” $\mid$ Expr “;” $\mid$ ID “=” Expr “;” $\mid$ Decl $\mid$ “if” “(” Expr “)” Stmt (“else” Stmt)? $\mid$
 “while” “(” Expr “)” Stmt $\mid$ “break” “;” $\mid$ “continue” “;” $\mid$ “return” Expr? “;” 语
 句块
 $\text{Block} \rightarrow$ “{” Stmt* “}” 函数定义
 $\text{FuncDef} \rightarrow$ (“int” $\mid$ “void”) ID “(” (Param(“,”
 Param)*)? “)” Block
 形参
 $\text{Param} \rightarrow$ “int” ID
 表达式
 $\text{Expr} \rightarrow \text{LOrExpr}$ 逻辑或表达式
 $\text{LOrExpr} \rightarrow$
 $\text{LAndExpr} \mid \text{LOrExpr}$ “||” LAndExpr 逻辑与表达式
 $\text{LAndExpr} \rightarrow \text{RelExpr} \mid \text{LAndExpr}$ “&&”
 RelExpr 关系表达式
 $\text{RelExpr} \rightarrow \text{AddExpr} \mid \text{RelExpr}$ (“<” $\mid$ “>” $\mid$ “<=” $\mid$ “>=” $\mid$ “==”
 $\mid$ “!=”) AddExpr 加减表达式
 $\text{AddExpr} \rightarrow \text{MulExpr} \mid \text{AddExpr}$ (“+” $\mid$ “-”) MulExpr 乘除模表达
 式
 $\text{MulExpr} \rightarrow \text{UnaryExpr} \mid \text{MulExpr}$ (“*” $\mid$ “/” $\mid$ “%”) UnaryExpr 一元表达式
 $\text{UnaryExpr} \rightarrow$
 $\text{PrimaryExpr} \mid$ (“+” $\mid$ “-” $\mid$ “!”) UnaryExpr 基本表达式
 $\text{PrimaryExpr} \rightarrow$ ID $\mid$ NUMBER $\mid$
 “(” Expr “)” $\mid$ ID “(” (Expr(“,” Expr)*)? “)” 编译单元
 声明
 常量声明
 变量声明
 语句
 语句块
 函数定
 义
 形参
 表达式
 逻辑或表达式
 逻辑与表达式
 关系表达式
 加减表达式
 乘除模表达式
 一元表达式
 基本表达式
 CompUnit Decl ConstDecl VarDecl Stmt Block FuncDef Param Expr LOrExpr LAndExpr RelExpr
 AddExpr MulExpr UnaryExpr PrimaryExpr $\rightarrow$ (Decl $\mid$ FuncDef) $^+ \rightarrow$ ConstDecl $\mid$ VarDecl $\rightarrow$ “const”
 “int” ID “=” Expr “;” $\rightarrow$ “int” ID “=” Expr “;” $\rightarrow$ Block $\mid$ “;” $\mid$ Expr “;” $\mid$ ID “=” Expr “;”
 $\mid$ Decl $\mid$ “if” “(” Expr “)” Stmt (“else” Stmt)? $\mid$ “while” “(” Expr “)” Stmt $\mid$ “break”
 “;” $\mid$ “continue” “;” $\mid$ “return” Expr? “;” $\rightarrow$ “{” Stmt* “}” $\rightarrow$ (“int” $\mid$ “void”) ID “(”
 (Param(“,” Param)*)? “)” Block $\rightarrow$ “int” ID $\rightarrow$ LOrExpr $\rightarrow$ LAndExpr $\mid$ LOrExpr “||” LAndExpr $\rightarrow$
 $\text{RelExpr} \mid \text{LAndExpr}$ “&&” $\text{RelExpr} \rightarrow$ AddExpr $\mid$ RelExpr (“<” $\mid$ “>” $\mid$ “<=” $\mid$ “>=” $\mid$ “==”
 $\mid$ “!=”) AddExpr $\rightarrow$ MulExpr $\mid$ AddExpr (“+” $\mid$ “-”) MulExpr $\rightarrow$ UnaryExpr $\mid$ MulExpr (“*”
 $\mid$ “/” $\mid$ “%”) UnaryExpr $\rightarrow$ PrimaryExpr $\mid$ (“+” $\mid$ “-” $\mid$ “!”) UnaryExpr $\rightarrow$
 ID $\mid$ NUMBER $\mid$ “(” Expr “)” $\mid$ ID “(” (Expr(“,” Expr)*)? “)”

ToyC 语言的终结符特征

标识符

终结符 **ID** 识别符合 C 规范的标识符。其正则表达式为： `[_A-Za-z][_A-Za-z0-9]*`

整数

终结符 **NUMBER** 识别十进制整数常量。其正则表达式为： `-?(0|[1-9][0-9]*)`

空白字符和注释

忽略空白字符和注释。空白字符包括空格、制表符、换行符等；注释的规范与 C 语言一致：单行注释以 `//` 开始、到最近的换行符前结束，多行注释以 `/*` 开始、以最近的 `*/` 结束）。

ToyC 语言的语义约束

程序结构

程序可以包含全局变量声明、全局常量声明和函数定义。程序必须包含一个名为 `main`、参数列表为空、返回类型为 `int` 的函数作为入口点。

函数

ToyC 中的函数支持传入若干 `int` 类型的参数，返回值可以为 `int` 或 `void` 类型。函数只能声明在全局作用域中且名称唯一，不能声明在函数体内。函数不能作为值来储存、传递或运算。函数调用必须写在被调函数声明之后（支持函数内部调用自身）。返回类型为 `int` 的函数必须在每一条可能的执行路径上通过 `return` 语句返回一个 `int` 类型的值。返回类型为 `void` 的函数可以不包含 `return` 语句，如果包含，则不能有返回值。

语句

ToyC 中的语句包括语句块、空语句、表达式语句、变量赋值、常量/变量声明、`if-else` 条件分支、`while` 循环、`break`、`continue` 和 `return`，它们的语法和语义与 C 语言一致：没有返回值的函数调用表达式不能作为 `if/while` 条件或赋值语句的右值；`break` 和 `continue` 只能出现在循环中等。

变量与常量声明

ToyC 支持声明 32 位有符号整数（`int`）类型的局部变量、全局变量以及局部/全局常量（`const`）。

- **初始化要求：**所有声明（`VarDecl` 和 `ConstDecl`）必须带有初始化表达式。
- **常量约束：**由 `const` 关键字修饰的标识符为常量。**常量的值在编译期必须能够确定**（其初始化表达式只能包含数字字面量、已被声明的常量以及由它们组合而成的算术/逻辑运算）。常量一旦声明，其值不能被修改（不能作为赋值语句的左值）。
- **作用域与生命周期：**全局变量和全局常量在整个程序运行期间均有效。局部变量/常量的生命周期从声明点开始，到所在作用域结束时结束。函数形参在函数体内可见。语句块 `Block` 会创建新的嵌套作用域，内层作用域会屏蔽外层作用域的同名变量、常量或形参。变量和常量的使用必须发生在声明之后。

表达式

ToyC 支持逻辑运算、关系运算、算术运算、括号表达式、变量/常量引用、字面值和函数调用等 C 语言中的常见表达式。运算符的优先级、结合性和计算规则等与 C 语言一致：逻辑与、逻辑或运算符遵循“短路计算”规则；非零视为“真”、零视为“假”；除数不能为零等。